

ERP Scanned Output — Qty Tracing Feasibility

Subject: 5-minute fetch of ERP accepted quantity, tagged by process sheet no.

Board: Output Tracking — ERP Scanned Output (/erp-scanned-output)

Date: 21 August 2026

Verdict. A 5-minute qty-delta poll is feasible. Reusing the full ERP sync on that cadence is not. The right slice is a WO-status-only read of accepted qty (quantum), compared to last known qty, writing a jump row only when the quantity increased.

1. What the board is actually doing today

The ERP Scanned Output page does not poll COMAIN. Opening or refreshing it only reads already-captured jump rows from planner_erp_qty_jump for the selected date range. Zero-change rows are already excluded from this view.

Detection (the fetch that notices a scan) happens only as a side-effect of ERP work-order sync. That is why many rows share one timestamp and are labelled “Detected on ERP sync”. The clock on the board is when planning noticed the change, not a shop-floor scan time from COMAIN.

Current production cadence:

- Full ERP sync: weekdays 08:00 and 13:00 (Windows Task Scheduler).
- Manual Sync ERP in the UI is the same pipeline.
- In-app ENABLE_AUTO_SYNC exists but is off by default (interval 15 minutes) because it adds COMAIN load and Supabase egress.

Observed board volume (07–20 Aug 2026 window): 27,171 accepted qty jumped, 873 jump events, 13 machines. That is roughly 67 recorded jumps per day under a twice-daily detect, because multiple scans between syncs are lumped into one jump.

2. How a jump is recorded (already implemented)

Key = process sheet no. (source_mps_no) + PP partial + stage.

1. Pull current accepted qty from WO status (total_acc_qty_produced).
2. Compare to last known qty in snapshot / previous jump.
3. First sighting of a sheet/stage is a baseline, not a jump.
4. If new – previous ≤ 0, ignore.
5. If increased, insert one row into planner_erp_qty_jump and tag machine from the live queue (or last exit / preferred machine).
6. Same-day duplicate of the same prev→new pair is dropped by unique constraint.

Quantity decreases are ignored. Two scans inside the same detect window collapse into one jump (today that window is hours; at 5 minutes it is usually one scan).

This is already the rule requested: no change in quantum → ignore; change → record, tagged by process sheet no. The missing piece is cadence and a lighter fetch, not new business logic.

3. Two different “queries” — do not mix them

Query	What it does	Run every 5 min?
-------	--------------	------------------

Dashboard API GET /api/erp-scanned-output	Reads planner_erp_qty_jump for the date range, groups by machine. Cheap. 8s statement timeout.	Harmless, but discovers no new scans.
Full ERP sync (all staging steps)	COMAIN → truncate/reload many tables, rebuild voucher cache, invalidate/warm planner caches, reconcile machine queues, then snapshot WO qty and capture jumps.	No. Locks Sync ERP, hammers COMAIN, makes the planner feel constantly mid-sync.
Existing WO-status step run_mfg_wo_status_sync	Heavy COMAIN join, then TRUNCATE + reload of entire mfg_wo_status, then rewrite daily snapshots for every stage — including unchanged qty.	Logically works, but more write traffic than tracing needs.
WO qty-only tracer (recommended)	Read current quantum from COMAIN WO tables, compare to last known in planner DB, insert jump only on increase. Do not truncate staging. Do not rebuild catalogs.	Yes, especially shop hours only.

4. What happens if a fetch keeps running in the background

If the job is the full ERP sync

- ~288 runs per day (every 5 minutes, 24h) or ~96 during an 8-hour shop day.
- Takes the ERP advisory lock: scheduled 08:00/13:00 and manual Sync ERP skip or wait.
- TRUNCATE of staging tables; brief empty/inconsistent reads for anything hitting those tables live.
- Cache invalidation + catalog warm on every tick — UI feels “just synced” all day.
- Queue reconcile after every tick.
- COMAIN under constant JOIN load. This is why auto-sync is off.

If the job is a dedicated qty tracer

- Most ticks write nothing (flat quantum is ignored).
- Jump count on the board goes up (09:12 scan and 11:40 scan become two rows, not one lumped 13:00 jump). Day total jumped qty stays the same.
- Table growth stays small: hundreds of jump rows per day, not millions.
- scanned_at becomes “noticed within 5 minutes”, still not true shop-floor scan time unless COMAIN exposes a scan timestamp (current WO query does not).
- If a tick overruns 5 minutes, skip the next one (own lock). Result is a gap, not a pile-up.
- Overnight/weekend ticks on an unscoped full WO extract waste COMAIN time for almost no jumps — gate to shop hours, or to in-process / on-machine jobs.

Do not keep rewriting daily snapshots the current way on a 5-minute timer. Today every in-scope WO stage is upserted on every sync, even when qty did not move. The table does not grow extra rows per day (one row per sheet/stage/date), but write volume would be huge. A 5-minute job should update last-known qty only when it changed.

5. Can WO status alone do the tracing?

Yes. That is the correct slice. You do not need PP vouchers, process-sheet info, shipped qty, SO detail, or cache rebuild.

One SQL on COMAIN cannot compute the deviation by itself. Current quantum lives in COMAIN

(mfg_wo_vch.total_acc_qty_produced). Last-known quantum lives in the planner DB (planner_erp_qty_jump / planner_erp_wo_qty_snapshot). The tracer is those two reads plus a compare.

Do not reuse run_mfg_wo_status_sync() for this job: it truncates the whole staging table. Query COMAIN, compare in the app (existing compute_qty_jumps / record_erp_qty_jumps), insert only increases.

Do not filter to execution_status = 'I' only. A scan that closes the lot can flip I → C in the same moment; that jump would be missed. Safer: all in-scope process sheets, or I/R plus any stage whose last known qty is still below required. Scoping to jobs currently on a machine lane (~13 machines) is even lighter.

6. Query 1 — current quantum from COMAIN (WO only)

Same join as the existing WO-status extract, stripped to process sheet + stage + qty. Keep the rework/scrap DISTINCT ON so a qty-1 scrap WO does not inflate quantum.

```
WITH wo_rows AS (
  SELECT
    t2.source_pp_no AS source_mps_no,
    COALESCE(NULLIF(t2.source_pp_partial_no, 0), 1) AS pp_partial_no,
    NULLIF(t2.stage_no::TEXT, '')::INTEGER AS stage_no,
    TRIM(COALESCE(t3.stage_desc, '')) AS stage_desc,
    t3.execution_status,
    t3.wo_qty_required,
    t3.total_acc_qty_produced,
    t3.total_rej_qty_produced
  FROM mfg_mps_vch t2
  JOIN mfg_wo_vch t3
    ON t2.wo_voucher_no = t3.voucher_no
   AND t2.stage_no = t3.stage_no
 WHERE t2.source_pp_no IS NOT NULL
   AND t2.stage_no IS NOT NULL
   AND (
     t2.source_pp_no LIKE '%NPS%'
    OR t2.source_pp_no LIKE '%APS%'
    OR t2.source_pp_no LIKE '%MPS%'
    OR t2.source_pp_no LIKE '%PPS%'
    OR t2.source_pp_no LIKE '%CPS%'
    OR t2.source_pp_no LIKE '%[SR]%'
   )
),
primary_wo AS (
  SELECT DISTINCT ON (source_mps_no, pp_partial_no, stage_no, stage_desc)
    source_mps_no, pp_partial_no, stage_no, stage_desc,
    execution_status, total_acc_qty_produced, total_rej_qty_produced
  FROM wo_rows
  ORDER BY source_mps_no, pp_partial_no, stage_no, stage_desc,
    wo_qty_required DESC NULLS LAST
)
SELECT
  source_mps_no,
  pp_partial_no,
  stage_no,
  stage_desc,
  SUM(total_acc_qty_produced) AS quantum,
  SUM(total_rej_qty_produced) AS rej_qty
FROM primary_wo
GROUP BY source_mps_no, pp_partial_no, stage_no, stage_desc;
```

7. Query 2 — deviation vs last known (planner DB)

After the COMAIN fetch, compare. Flat quantum is dropped; only an increase is written.

```
SELECT
  c.source_mps_no,
  c.pp_partial_no,
  c.stage_no,
  p.acc_qty_produced AS prev_qty,
  c.quantum          AS new_qty,
  c.quantum - p.acc_qty_produced AS jump
FROM current_wo_qty c
JOIN last_known_qty p
  ON p.source_mps_no = c.source_mps_no
 AND p.pp_partial_no = c.pp_partial_no
 AND p.stage_no      = c.stage_no
WHERE c.quantum > p.acc_qty_produced + 0.000001;
```

last_known_qty is already what jump capture reads (latest snapshot or latest jump per sheet/stage). First sighting of a sheet is a baseline, not a jump — same as the board today.

8. Recommended operating design

1. Leave weekday 08:00 / 13:00 full ERP sync as the catalog and status refresh. Do not put that pipeline on a 5-minute timer.
2. Add a narrow background job (Windows task or in-app thread), shop hours only (e.g. 07:00–19:00 weekdays).
3. Pull accepted qty from COMAIN for in-scope process sheets / stages (Query 1). Prefer jobs on a machine lane if that is enough coverage.
4. Compare to last qty (Query 2 / existing record_erp_qty_jumps).
5. No change → exit. Increase → insert one jump row. Do not TRUNCATE mfg_wo_status. Do not invalidate caches.
6. Give this job its own lock so it does not block Sync ERP. If a tick is still running, skip the next.
7. Update last-known qty only when it changed.

9. Code / table references

Item	Location
Jump capture + ignore-if-flat	planning/erp_scanned_output_service.py — compute_qty_jumps, record_erp_qty_jumps
Dashboard read	planning/erp_scanned_output_route.py — GET /api/erp-scanned-output
WO snapshot after sync	planning/erp_actuals.py — record_erp_wo_qty_snapshots
COMAIN WO extract	sync.py — _build_mfg_wo_status_sql, run_mfg_wo_status_sync
Jump table	public.planner_erp_qty_jump (migrations/add_erp_qty_jump.sql)
Daily snapshot table	public.planner_erp_wo_qty_snapshot (one row per sheet/stage/date)
Unique jump key	(source_mps_no, pp_partial_no, stage_no, prev_acc_qty, new_acc_qty, scanned_date)
Sync cooldown	300 seconds in sync.py (bypassed when force=True)

10. Bottom line

- The ignore/record logic is already sound and tagged by process sheet no.
- The board query you were viewing is history only; it cannot trace new scans.
- A 5-minute WO-status quantum tracer is feasible if it is a read + compare, not the ERP sync already run twice a day.
- If that loop is the full sync, the background effect is lock contention, cache churn, and a constantly busy planner — not more accurate output tracking.

Machine Planning — ERP Scanned Output feasibility note. Prepared 21 August 2026. This note describes current behaviour and a recommended tracer; it does not change production scheduling.